



Agentic Development Framework for D365 BC

**Simple Enough for Functional Consultants.
Robust Enough for Pro Developers.**

Created by AJ Ansari, Microsoft MVP

The Playbook — four phases, twelve steps, twelve gates

Shapes and roles

SHAPE LEGEND

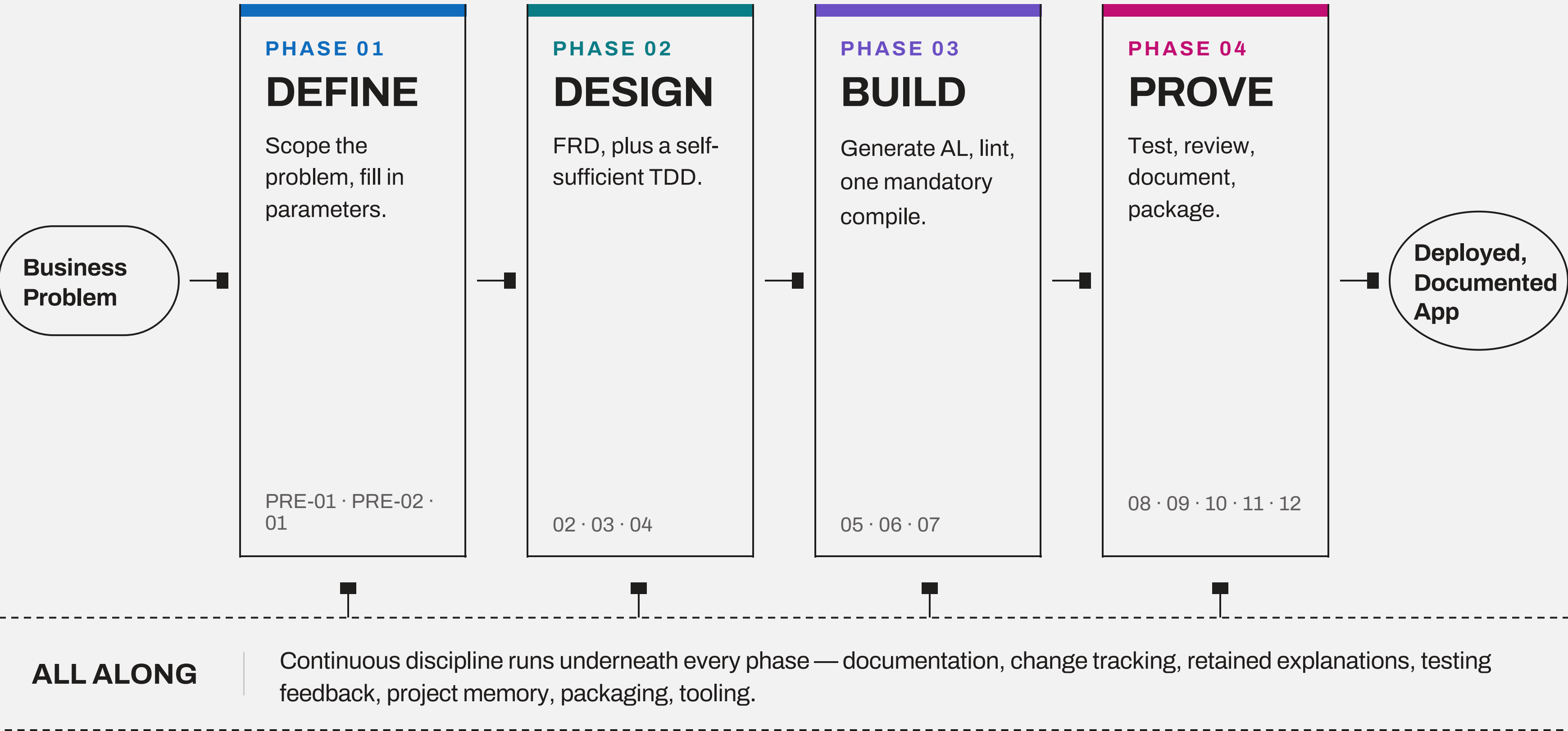
Start / end	Start or end point of a flow
Action	An action, step, or process
Doc	Input or output — data, a document
Exit gate	An exit gate — the checkpoint that must be met
Choice?	A decision point
All along	A cross-cutting concern, running underneath

ROLE LEGEND — §1.7

	Main role Generates and edits all code; owns the continuity documents.
	Light role Post-generation pre-flight only. Reports findings, never edits.
	Reasoning role Authors, reviews, diagnoses. Never edits code or continuity docs.

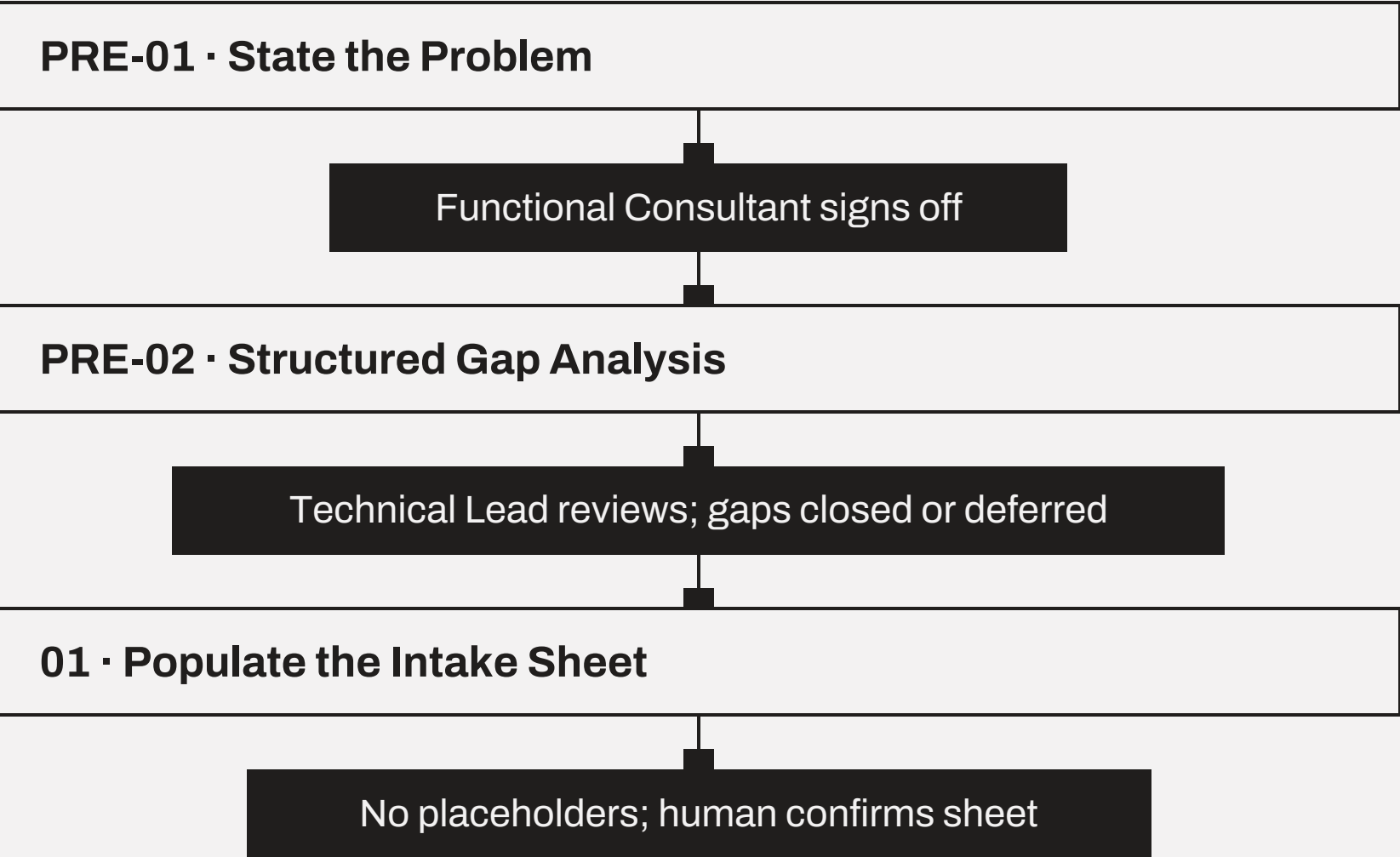
Where §1.7 role assignment isn't configured, every colored step is done by the single running model instead.

The whole framework in one picture



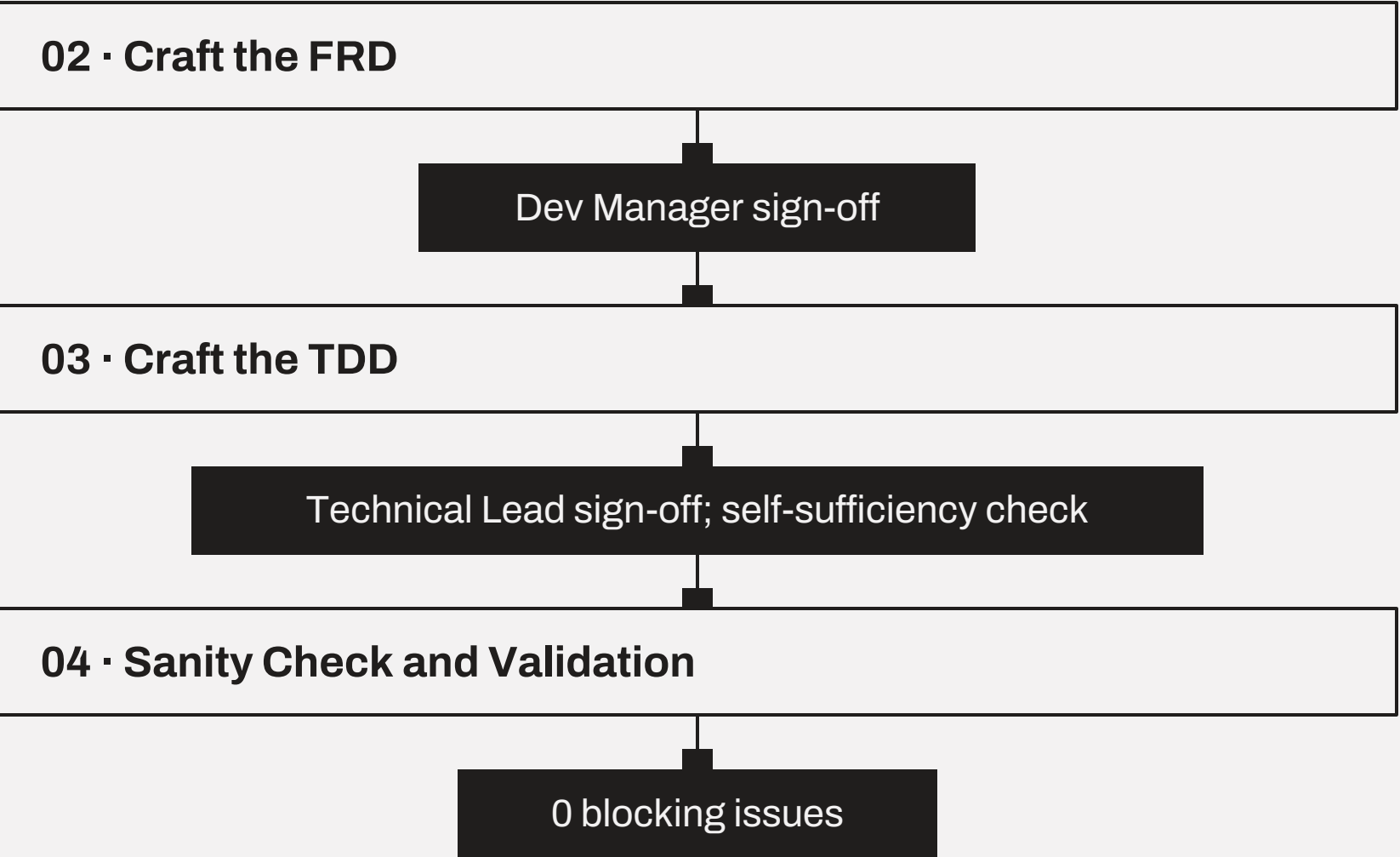
Every step, and the gate that closes it

DEFINE



↓ to DESIGN, Step 02

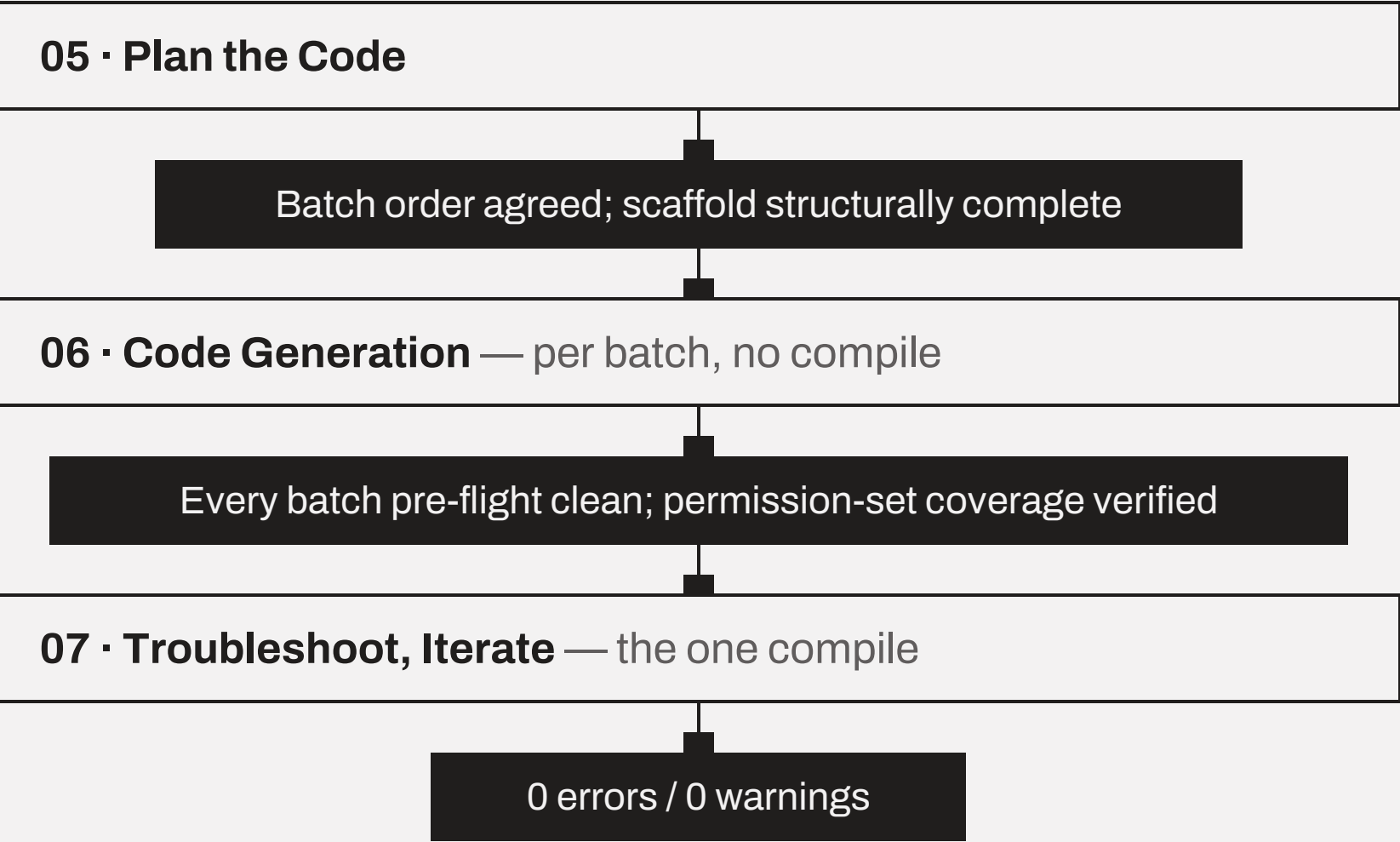
DESIGN



↓ to BUILD, Step 05

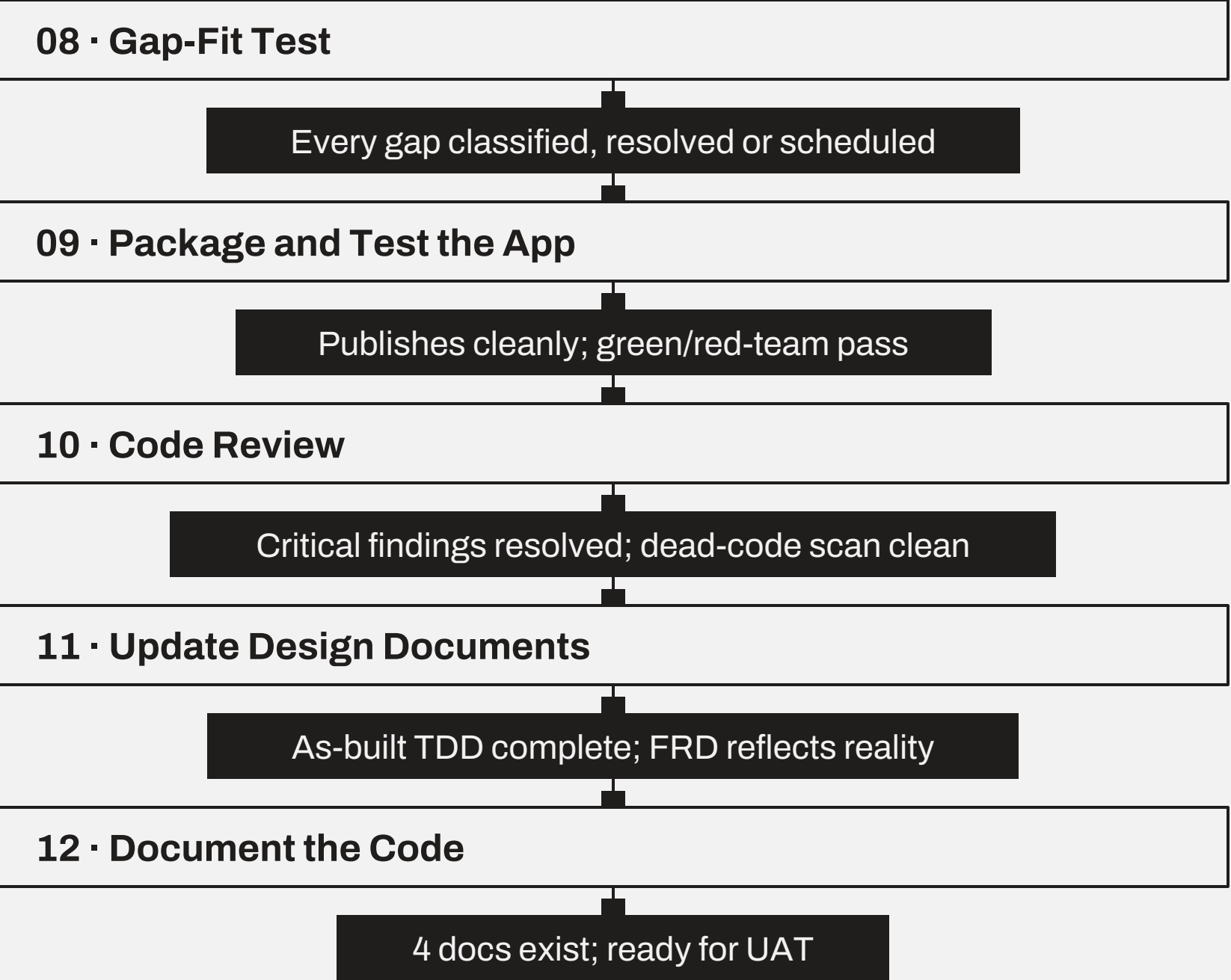
Every step, and the gate that closes it

BUILD



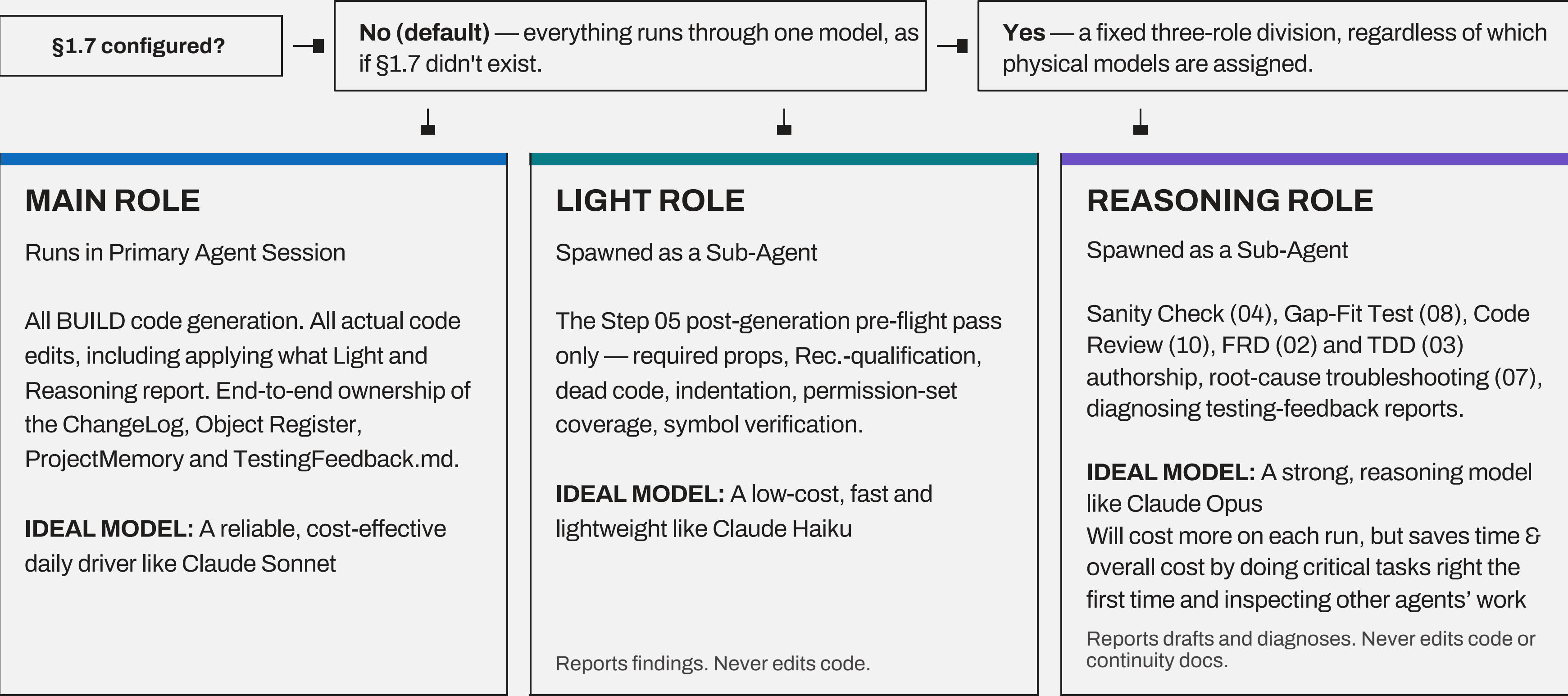
↓ to PROVE, Step 08

PROVE



SCHEMATIC 3 — CROSS-CUTTING

Model & Effort Assignment (§1.7)



Light and Reasoning relay findings to Main

The codebase & continuity documents

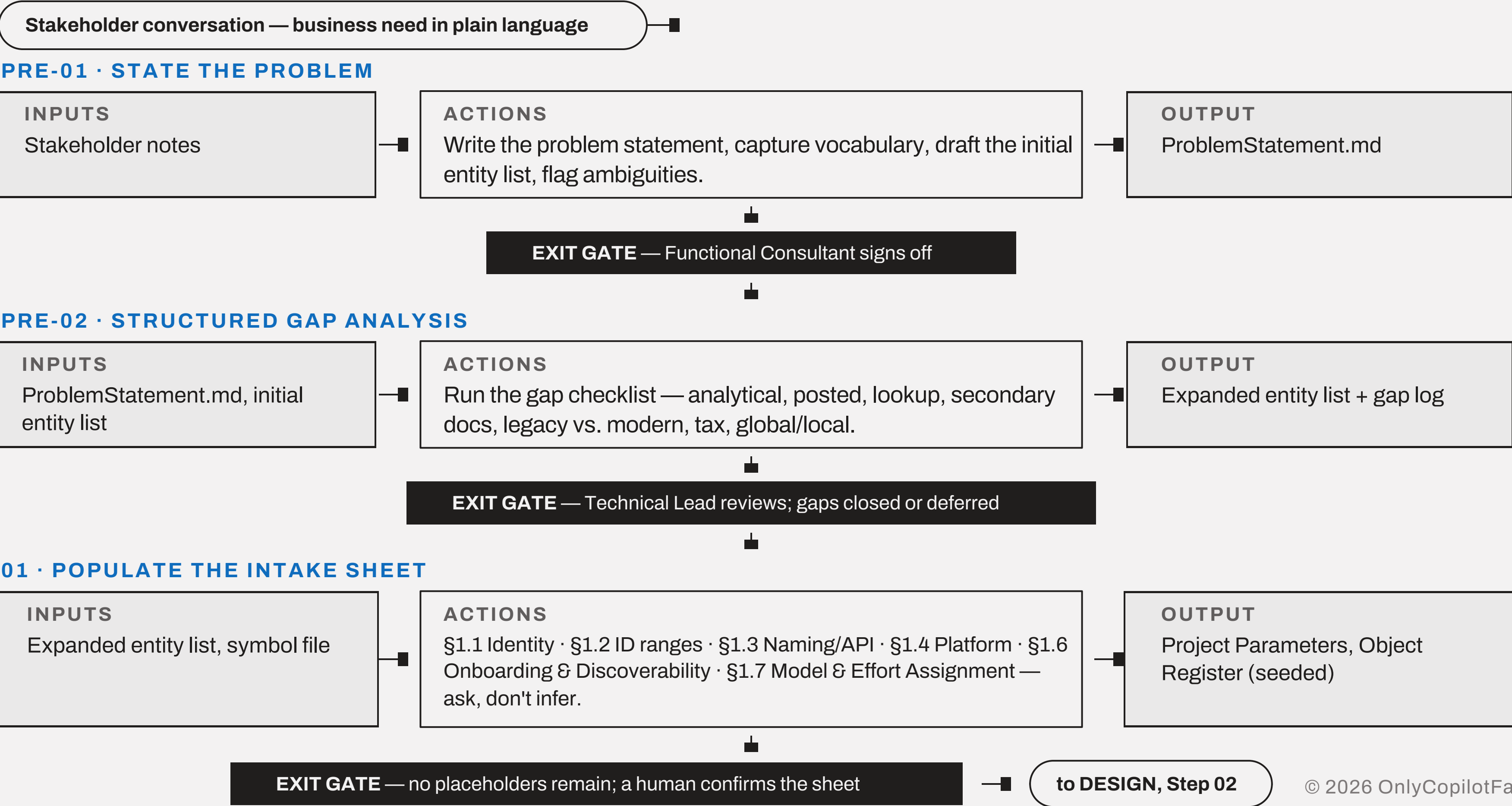
PHASE 01

DEFINE

Scope the problem and fill in the parameters. Ask, don't infer.

PRE-01 State the Problem · PRE-02 Structured Gap Analysis · 01 Populate the Intake Sheet

From stakeholder conversation to confirmed parameters



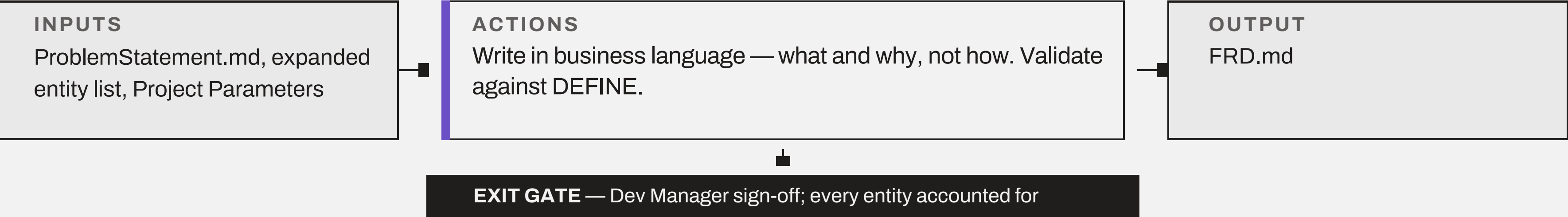
DESIGN

An FRD in business language, and a TDD complete enough to build from alone.

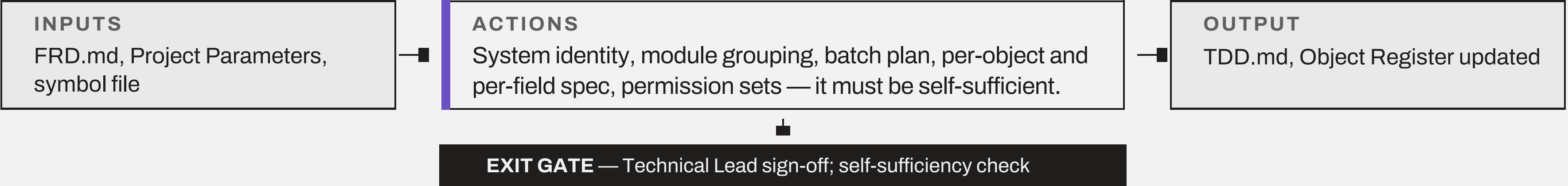
The Main role integrates every draft — saves the file, keeps ChangeLog and ProjectMemory bookkeeping — and carries it to the human for sign-off. The Reasoning role never owns sign-off.

Write what, then how — and check they agree

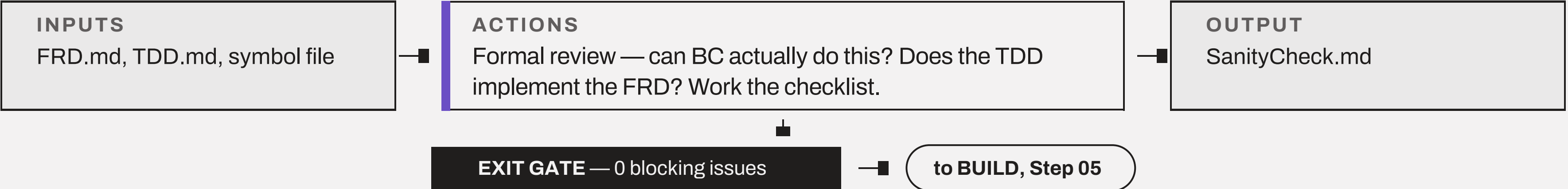
02 · CRAFT THE FRD REASONING ROLE



03 · CRAFT THE TDD REASONING ROLE



04 · SANITY CHECK AND VALIDATION REASONING ROLE

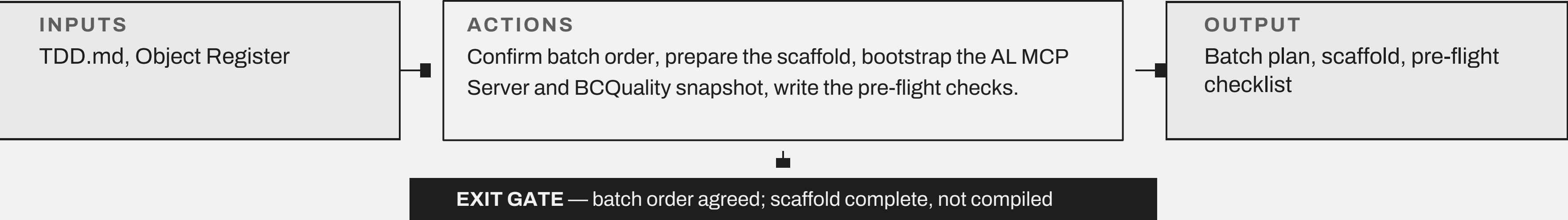


BUILD

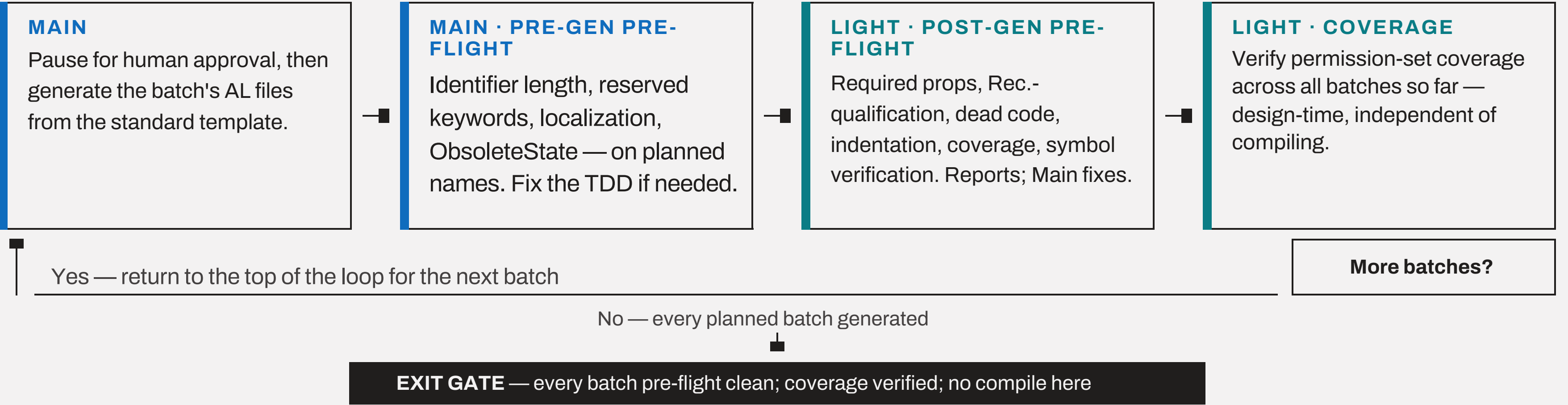
No per-batch compile. Symbol-verified lint instead, and exactly one mandatory compile once every planned batch exists.

Plan the batches, then generate them — without compiling

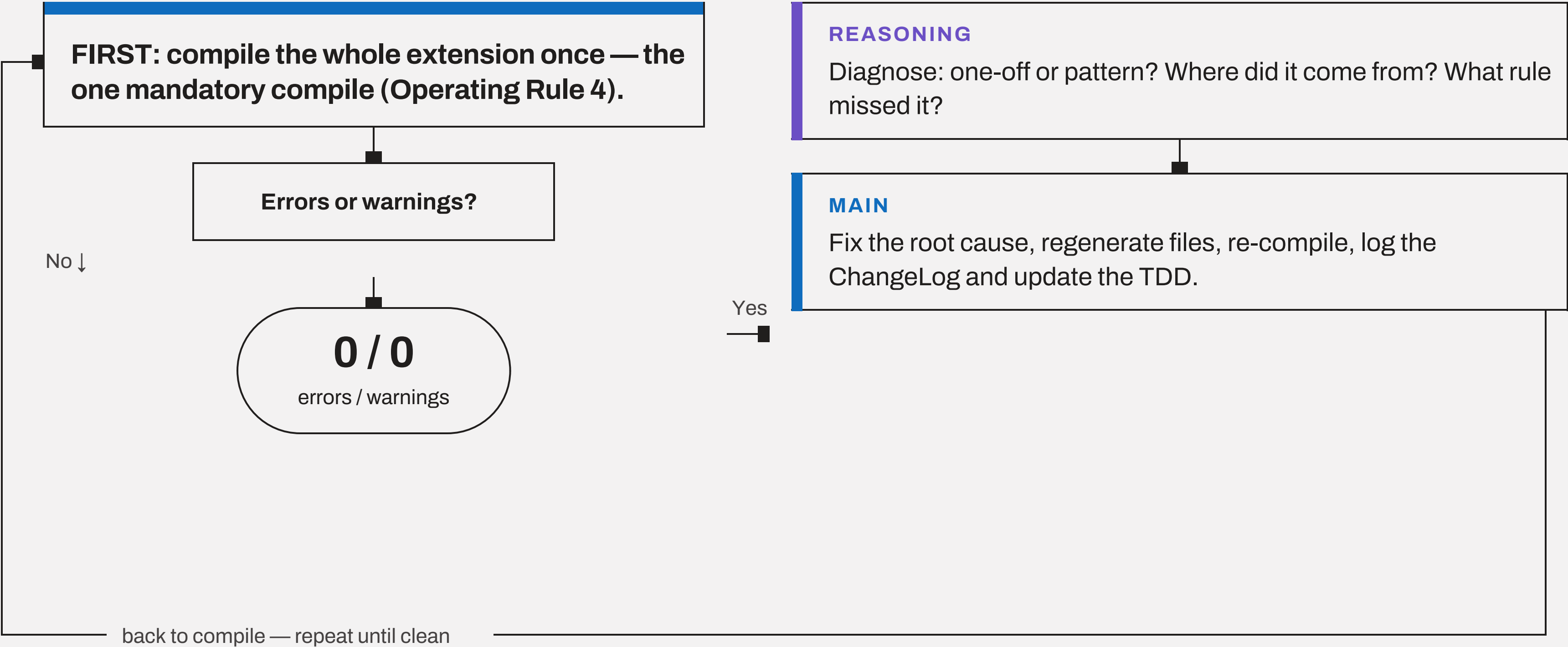
05 · PLAN THE CODE



06 · CODE GENERATION repeats once per batch



Troubleshoot, iterate — the one mandatory compile



EXIT GATE — full extension compiles clean; ChangeLog and TDD reconciled

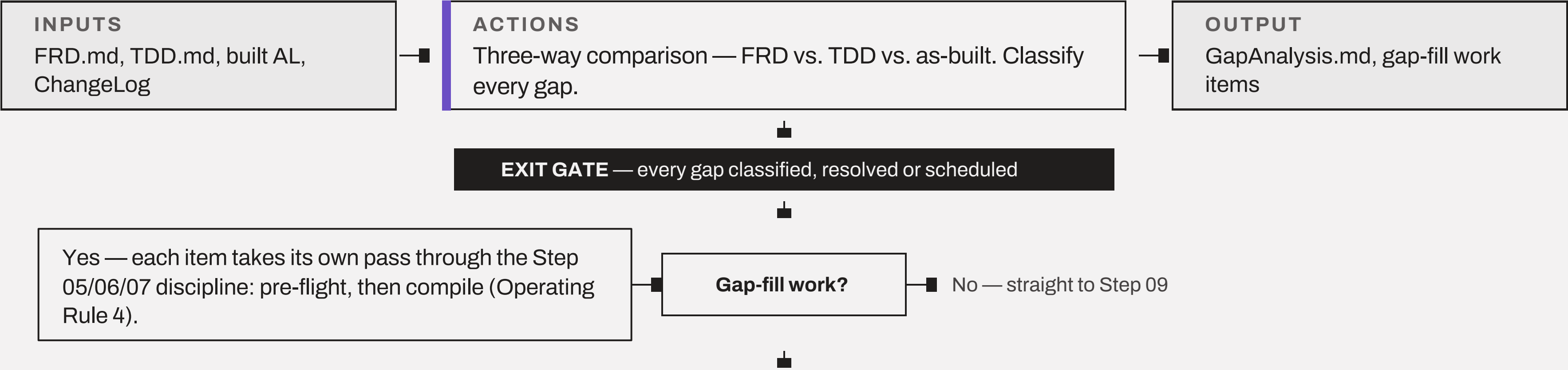
PROVE

Test, review, document, package — until it is ready for UAT.

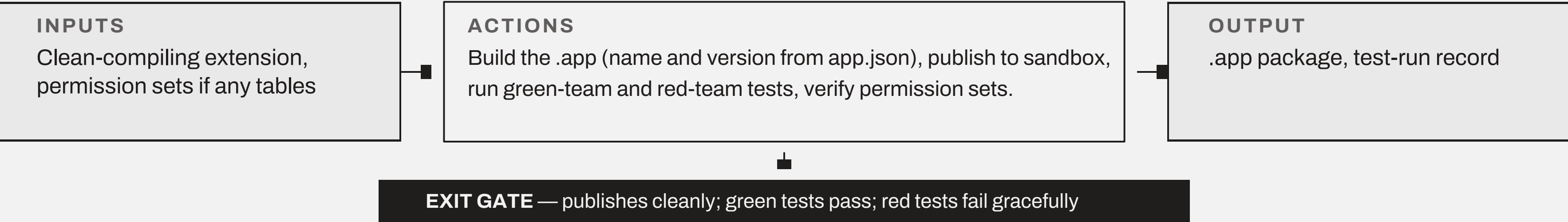
Does the built thing match what was promised?

08 · GAP-FIT TEST, FIDELITY VALIDATION

REASONING ROLE



09 · PACKAGE AND TEST THE APP

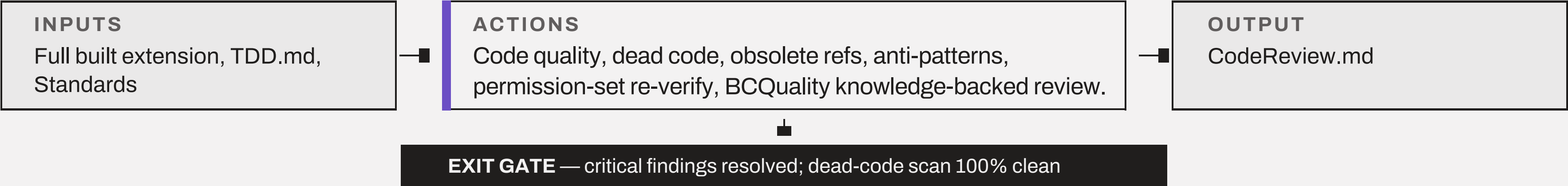


PHASE 04 — PROVE · SCHEMATIC 4.4, STEPS 10–12

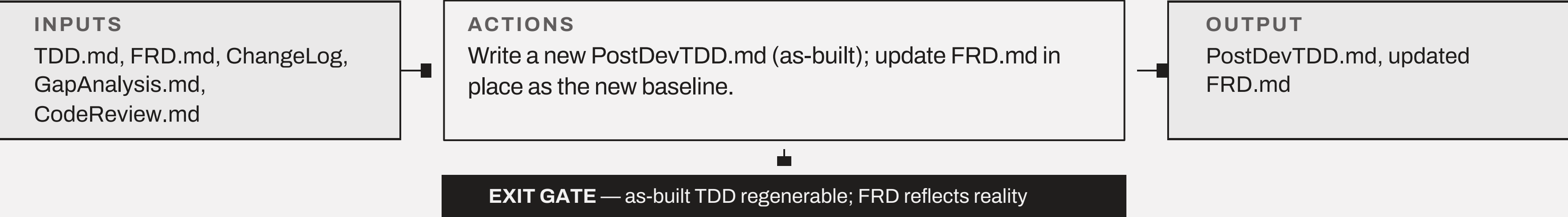
Review it, then leave the documentation behind

10 · CODE REVIEW

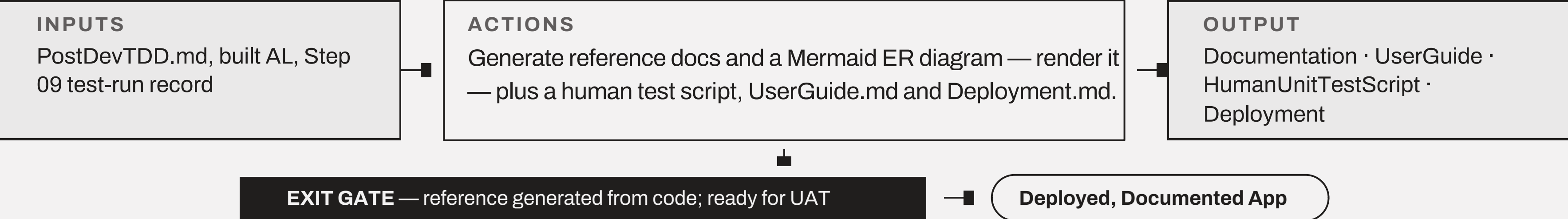
REASONING ROLE



11 · UPDATE DESIGN DOCUMENTS

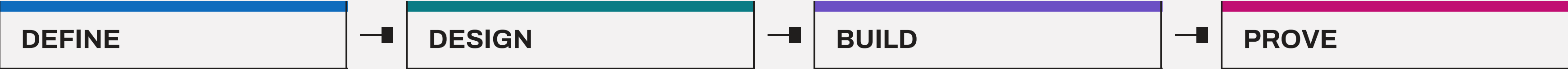


12 · DOCUMENT THE CODE



SCHEMATIC 5 — CROSS-CUTTING

ALL ALONG — continuous discipline



ALL ALONG EVERY PHASE, EVERY STEP			
Document Keep every required doc current, not retroactively.	Track Changes Every FRD/TDD deviation logged before the next batch.	Retain Explanations Who decided, and why — not just the outcome.	Testing Feedback Log Verbatim findings, triaged explicitly, cross-referenced.
Project Memory docs/ProjectMemory.md — an anchor, not a narrative.	Packaging & Versioning Never delete a package; propose bumps, don't apply silently.	AL MCP Server Bootstrap once; prefer its tools over ad hoc terminal use.	BCQuality Snapshot Fetch once; refresh only on explicit request.

Simple Enough for Functional Consultants. Robust Enough for Pro Developers.

Created by AJ Ansari, Microsoft MVP